

Understanding Pod Lifecycle and Pod States in Kubernetes

Pod Lifecycle in Kubernetes – Overview

A Pod is the smallest deployable unit in Kubernetes.

When a Pod is created, it does not immediately start running. Kubernetes moves the Pod through different phases before it becomes operational.

Understanding the Pod Lifecycle helps DevOps engineers monitor applications, troubleshoot issues, and identify failures quickly.

This documentation covers:

- What is Pod Lifecycle?
 - Pod Lifecycle flow.
 - Different Pod States.
 - Common Pod Error States.
 - Troubleshooting Commands.
 - Creating a Pod and observing its lifecycle.
 - Verifying Pod status.
 - Viewing Pod details and logs.
 - Accessing the Pod terminal.
 - Deleting the Pod.
 - Interview-focused concepts.
-

What is Pod Lifecycle?

Pod Lifecycle refers to the journey of a Pod from creation to termination.

Whenever a Pod is created, Kubernetes performs multiple operations such as:

- Scheduling the Pod.
- Selecting a Node.
- Pulling container images.

- Creating the container.
- Running the application.

Depending on the result of these operations, the Pod moves through different states.

Pod Lifecycle Flow

```
Pending
  ↓
ContainerCreating
  ↓
Running
  ↓
Succeeded / Failed
```

In case of issues:

```
ErrImagePull
ImagePullBackOff
CrashLoopBackOff
Unknown
```

Pod States

1. Pending

The Pod has been accepted by Kubernetes but is not yet running.

Kubernetes is performing:

- Node selection.
- Pod scheduling.
- Resource allocation.

Example:

NAME	READY	STATUS
nginx-pod	0/1	Pending

2. ContainerCreating

The Pod has been scheduled to a Node.

Kubernetes is now:

- Pulling the container image.
- Creating networking.
- Mounting volumes.
- Starting the container.

Example:

NAME	READY	STATUS
nginx-pod	0/1	ContainerCreating

3. Running

The container has started successfully and the Pod is operational.

Example:

NAME	READY	STATUS
nginx-pod	1/1	Running

Important:

Running does not always mean the application is healthy.

Application health is checked using:

- Liveness Probe.
 - Readiness Probe.
 - Startup Probe.
-

4. Succeeded

The Pod completed its task successfully and exited with Exit Code 0.

Common examples:

- Batch Jobs.
- One-time scripts.
- Scheduled tasks.

Example:

STATUS = Succeeded

5. Failed

The Pod terminated with a non-zero exit code.

Possible reasons:

- Application crash.
- Invalid command.
- Script failure.

Example:

```
STATUS = Failed
```

6. Unknown

Kubernetes cannot determine the current status of the Pod.

Usually happens when communication between the Control Plane and Node is lost.

Example:

```
STATUS = Unknown
```

Common Pod Error States

These states are frequently asked in interviews.

ErrImagePull

Kubernetes attempted to pull the image but failed.

Possible reasons:

- Wrong image name.
- Private image without credentials.
- Registry connectivity issues.

Example:

```
image: nginx12345
```

Output:

```
ErrImagePull
```

ImagePullBackOff

Kubernetes repeatedly attempts to pull the image but continues to fail.

Usually occurs after ErrImagePull.

Flow:

```
ErrImagePull
  ↓
ImagePullBackOff
```

CrashLoopBackOff

The container starts successfully but crashes repeatedly.

Kubernetes continuously restarts the container.

Flow:

```
Start
  ↓
Crash
  ↓
Restart
  ↓
Crash
  ↓
Restart
```

Common reasons:

- Application startup failure.
 - Invalid configuration.
 - Missing environment variables.
 - Dependency issues.
-

Troubleshooting Commands

Check Pod Status

```
kubectl get pods
```

Detailed Pod Information

```
kubectl describe pod nginx-pod
```

View Pod Logs

```
kubectl logs nginx-pod
```

Access Pod Terminal

```
kubectl exec -it nginx-pod -- /bin/sh
```

Practical Exercise

Step 1: Create a YAML File

Create a file named:

pod-lifecycle.yaml

Command:

```
vim pod-lifecycle.yaml
```

Step 2: Write the YAML Configuration

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
  containers:
    - name: nginx-container
      image: nginx
```

Step 3: Create the Pod

```
kubectl apply -f pod-lifecycle.yaml
```

Expected Output:

```
pod/nginx-pod created
```

Step 4: Verify the Pod Creation

```
kubectl get pods
```

Possible Output:

```
Pending
```

or

```
ContainerCreating
```

or

```
Running
```

Step 5: Watch the Pod Lifecycle in Real Time

```
kubectl get pods -w
```

Example:

NAME	READY	STATUS
nginx-pod	0/1	Pending
nginx-pod	0/1	ContainerCreating
nginx-pod	1/1	Running

Step 6: Inspect Pod Details

```
kubectl describe pod nginx-pod
```

Verify:

- Pod IP.
 - Node Name.
 - Events.
 - Container Status.
 - Image Name.
-

Step 7: Check Pod Logs

```
kubectl logs nginx-pod
```

If the application generates logs, they will be displayed.

Step 8: Access the Pod Terminal

```
kubectl exec -it nginx-pod -- /bin/sh
```

Exit:

```
exit
```

Step 9: Delete the Pod

```
kubectl delete pod nginx-pod
```

Verify deletion:

```
kubectl get pods
```

Expected Output:

```
No resources found
```

Error Practice

Simulating ImagePullBackOff

Create a file named:

bad-pod.yaml

```
apiVersion: v1
kind: Pod

metadata:
  name: bad-pod

spec:
  containers:
    - name: nginx
      image: nginx-invalid-image
```

Create the Pod:


```
kubectl apply -f bad-pod.yaml
```

Check Status:

```
kubectl get pods
```

Expected:

```
ErrImagePull
```

Later:

```
ImagePullBackOff
```

Inspect the error:

```
kubectl describe pod bad-pod
```

Observe the Events section.

Interview Questions

Difference Between Pending and ContainerCreating

Pending:

- Pod is not yet assigned to a Node.
- Scheduling is in progress.

ContainerCreating:

- Node has already been assigned.
 - Container setup process has started.
-

Difference Between ErrImagePull and ImagePullBackOff

ErrImagePull:

- First image pull attempt failed.

ImagePullBackOff:

- Kubernetes keeps retrying image download with increasing delay.
-

What is CrashLoopBackOff?

CrashLoopBackOff is a condition where a container continuously crashes and Kubernetes repeatedly restarts it.

Pod Lifecycle Summary Table

State	Meaning
Pending	Waiting for scheduling/resources
ContainerCreating	Container setup in progress
Running	Container running successfully
Succeeded	Task completed successfully
Failed	Task failed
Unknown	Status cannot be determined
ErrImagePull	Image pull failed
ImagePullBackOff	Repeated image pull failure
CrashLoopBackOff	Continuous crash and restart

Key Learnings

By completing this lab, we learned:

- Pod Lifecycle.
- Pod States.
- Pod Scheduling Process.
- Container Creation Process.
- Common Failure States.
- Troubleshooting Commands.

- Real-Time Pod Monitoring.
- Error Simulation and Analysis.
- Interview-Oriented Concepts.